

EnCodecMAE: leveraging neural codecs for universal audio representation learning

Leonardo Pepino^{1,2}, Pablo Riera^{1,2}, Luciana Ferrer¹

¹Instituto de Investigación en Ciencias de la Computación (ICC), CONICET-UBA, Argentina

²Departamento de Computación, FCEyN, Universidad de Buenos Aires (UBA), Argentina

lpepino@dc.uba.ar, priera@dc.uba.ar, lferrer@dc.uba.ar

Abstract

Universal audio representation learning aims to obtain foundational models that are useful for diverse tasks involving speech, music and environmental sounds. To achieve this, often methods inspired by self-supervised models from NLP (e.g., BERT), and vision, like masked autoencoders (MAE), are adapted to the audio domain. In this work, we explore the use of EnCodec, a neural audio codec, to generate discrete targets for a MAE-based universal audio model. We evaluate our approach, EnCodecMAE, across various tasks and find that, on average, it outperforms state-of-the-art audio representation models. Moreover, we analyze the impact of several factors on downstream performance, concluding that increasing model size leads to performance improvements, that the optimal input representation depends on the type of task, that self-training is beneficial and that diversity in the training dataset is essential to achieve good performance across different audio tasks.

Index Terms: audio representations, self-supervised, transformer, music, speech

1. Introduction

The search for universal audio models that can be used to solve a wide range of tasks related not only to speech, but also to music and environmental sounds, has received increased attention in recent years [1, 2, 3, 4, 5, 6, 7]. Some works pretrain these models in a supervised way using existing large labeled datasets like Audioset [8, 9]. Other works use self-supervised learning (SSL), which is an effective way to leverage large unlabeled audio datasets. SSL methods resort to learning a pretext task, like masked language modelling, which consists of predicting masked parts of an utterance [10, 11] or contrasting the output on a masked time step with the correct output and a set of distractors taken from the same utterance [12]. These pretext tasks operate at the time or time-frequency patch level and learn relationships between parts of an utterance. Contrastive learning has also been used to learn instance-level embeddings, by creating positive examples for a certain audio through augmentation techniques and negative examples by extracting fragments from different audio samples [13, 14]. BYOL-A [15] is another recently proposed method to learn instance-level representations of audio signals without requiring negative examples, by minimizing the distance between two augmented views of a spectrogram encoded by a teacher and a student network.

The pretrained upstream models can be adapted to downstream tasks using relatively small labelled datasets by feeding the activations from one or more of its layers to a downstream network. The pretrained model can be frozen [2, 16] or fine-tuned [1, 4, 7]. When the task of interest has instance-level labels but the pretrained model generates frame-level represen-

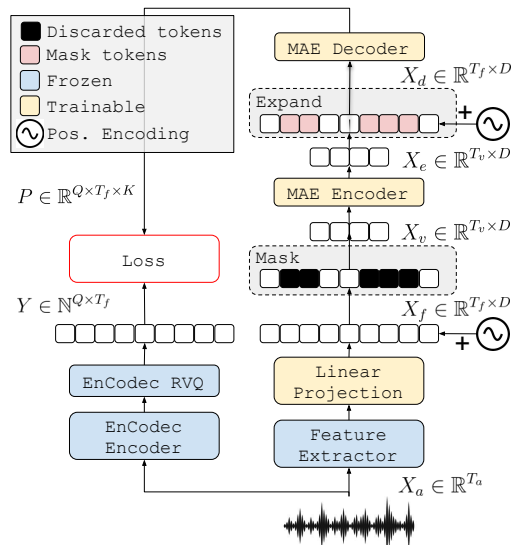


Figure 1: Illustration of the EnCodecMAE architecture, consisting of a masked autoencoder trained to predict the discrete representation Y of the unmasked audio X_a given by EnCodec.

tations, an operation that pools over time is usually applied to generate instance-level representations [1, 2, 4].

Our work follows the ideas originally proposed in BERT [17] for NLP and then adapted for speech in HuBERT [10] and DiscreteBERT [11], which consist of masking regions from the input signal and learning to predict discrete targets corresponding to the masked regions. In HuBERT the discrete targets are obtained from k-means clustering of MFCCs and, in a second stage, from internal representations. BEATs [7] uses a similar approach as HuBERT, but the outputs of a random quantizer are used as initial targets and iteratively refined. In this work, we propose to use the discrete units learned by EnCodec [18], a general audio neural codec, as targets for our model, and then introduce, as in HuBERT, an additional target by clustering internal representations. We use the Masked Autoencoder (MAE) architecture [19], which enables efficient pretraining. Our work is novel in several aspects: (i) a neural audio codec (EnCodec) is used to generate discrete targets to learn universal audio representations; (ii) a MAE architecture is applied over time instead of rectangular time-frequency patches, achieving a performance in environmental tasks comparable to the best patch-based MAEs in the literature, while providing higher time resolution and improving performance in speech tasks; (iii) the performance on an ASR task is reported, something that, as far as we know, was never done for a univer-

sal audio model. As shown in the results section, EnCodecMAE achieves the best global performance when compared with various state-of-the-art universal audio representations.

2. Proposed model

In this work we propose EnCodecMAE, a masked autoencoder that uses EnCodec representations as targets during learning. The EnCodecMAE architecture is depicted in Figure 1. Features X_f are extracted from an audio signal X_a with length T_a and projected to a dimensionality D . We compare the use of melspectrograms and outputs from the EnCodec encoder as input features. In both cases, 75 embeddings per second are generated, resulting in a sequence of length T_f . Similarly to wav2vec 2.0 and HuBERT, a proportion M_{prop} of the embeddings is randomly masked, sampling the starting indices for the mask without replacement, and masking the subsequent M_{gap} time steps for every sampled index. In contrast to wav2vec 2.0 and HuBERT, the masked embeddings are discarded from the sequence as in [19], instead of replaced by a mask token. As a consequence, the resulting sequence X_v , which contains only the visible embeddings, has a shorter length T_v than the original one. X_v is passed through an encoder and then expanded to the original size T_f by inserting mask tokens in the masked regions. Sinusoidal positional embeddings are added again to inform the decoder of the position of the masked regions that need to be reconstructed, and the resulting sequence X_d is input to a decoder. The model is trained to minimize a weighted cross entropy loss between the posteriors P output by the decoder and the discrete target, Y , generated by the EnCodec residual vector quantization (RVQ) layer. The following sections explain the components of the model. Source code is available at <https://github.com/habla-liaa/encodecmae>.

2.1. EnCodec

EnCodec [18] is a neural audio codec consisting of an autoencoder with a quantized latent space. EnCodec achieves a high compression rate while also minimizing the perceptible distortion of the decoded audios, obtaining higher MUSHRA scores than hand-crafted codecs and Soundstream [20] using the same bitrate, both for speech and music signals. This suggests that the quantized latent space from EnCodec contains most of the perceptually-relevant information for reconstructing not only speech, but also more complex signals like music.

The EnCodec architecture consists of a CNN encoder with 4 convolutional blocks that downsample the audio signal from 24kHz to 75 Hz, followed by 2 LSTM layers and a final convolutional layer. The encoder’s output is a sequence of 128-dimensional vectors, which are passed through a residual vector quantization (RVQ) block with 32 codebooks, each with 1024 possible values. RVQ maps the input to the closest entry in the first codebook. Then, the residual is computed and mapped to the closest entry of the second codebook. This process is repeated for every codebook, increasingly refining the quantized outputs. Finally, the decoder mirrors the encoder architecture, upsampling the encoded signal with transposed convolutions and recovering the original audio.

2.2. Masked autoencoders

Masked autoencoders (MAE) were proposed in [19] as a self-supervised model for computer vision and an alternative to Vision Transformers (ViT), and have been recently applied to audio representation learning [4, 5, 6, 21], using spectrogram

patches as input and as targets during training. They are trained in a similar way to BERT, by masking parts of the input signal and predicting the masked parts from the unmasked ones. In contrast to BERT, which replaces the masked regions with a mask token, MAEs discard the masked regions before feeding the sequence to the encoder leading to a more efficient training.

As usually done with transformers, positional embeddings are added to the encoder input. Importantly, this is done before masking so that the remaining embeddings carry information on their absolute location in the sequence. The encoder and decoder networks consist of a stack of transformer layers. Before decoding, the encoder’s output is expanded to its original length by adding mask tokens learned during training as a shared embedding. The decoder has a final classification layer with softmax activation. This layer predicts the discrete targets from the EnCodec’s RVQ, corresponding to neural codec representations of the input audio. This is subtly different from the original MAE as the decoder is not designed to output an estimate of the encoder’s input before masking, but a discretized version of it. We use the asymmetric encoder-decoder architecture proposed in [19], with a small decoder and a larger encoder. In our setup, for the encoder we use 5 transformer layers in the small model, 10 in the base, and 20 in the large. All our models use only 2 layers in the decoder as in [19] and pre-post layer normalization [22]. The dimension of the embeddings D is 768 for the small and base models, and 1024 for the large model.

2.3. Training procedure

The training procedure consists of two stages. Initially, the targets for the model are generated by the EnCodec RVQ layer. These consist of an index $y_q(t)$ between 1 and 1024 for each time step t and codebook q . The loss function is a weighted cross-entropy, with tunable weights for masked versus unmasked time steps and each individual codebook:

$$L = \sum_{q=0}^Q \gamma_q \left[\alpha \sum_{t \in M} C(y_q(t), P_q(t)) + \beta \sum_{t \notin M} C(y_q(t), P_q(t)) \right]$$

where $P_q(t)$ is the vector of 1024 posteriors generated by the decoder for time step t and codebook q , M is the set of masked indices, and $\alpha = \delta/|M|$ and $\beta = (1 - \delta)/(T_f - |M|)$ are weights given to the masked and unmasked regions, respectively, with δ being a tunable parameter. The loss corresponding to each codebook stream q is, in turn, weighted by γ_q . The loss for each codebook and time step is given by $C(y_q(t), P_q(t)) = -\log(P_{q,y_q(t)}(t))$. Finally, after 500k training steps, a self-training stage is performed. We extract embeddings from the last encoder layer for 10k randomly-sampled audio signals and train a k-means model with $k = 1024$. The model is trained for 150k additional steps to predict these discrete targets. As in HuBERT, for the large model we perform k-means on activations from the base model after self-training.

3. Experimental setup

3.1. Pretraining

For pretraining we used a mixture of datasets composed of Audioset (AS) [23], Free Music Archive (FMA) [24] and the 6000-hour Libri-Light split (LL6K) [25]. Audioset contains over 2 million 10-second audio clips from YouTube videos covering 527 different audio events. We downloaded more than 1.6M audios (around 4500 h). FMA consists of 106,574 30-second

tracks covering 161 genres for a total of over 800 hours of music. Libri-Light consists of english speech from audiobooks.

In all our experiments we trained with 4-second segments by randomly cropping the audio samples. At inference time, longer audios are chunked to 4 seconds and the resulting embeddings are concatenated. Melspectrograms are calculated using a Hanning window with 640 samples, a hop size of 320 samples and 256 filters, matching EnCodec rate. Our models were trained for 500k steps using AdamW optimizer with a weight decay of 0.05, β_1 of 0.9, β_2 of 0.95, a fixed learning rate of $1e-4$, and a batch size of 128. EnCodec’s parameters were downloaded from Hugging Face¹ and are not changed during training. We used 8 of the 32 quantizers available in EnCodec as targets for training. We selected the training hyperparameters based on a base model’s performance on HEAREval’s validation data, after 150k training steps, resulting in: $\delta = 0.9$, $M_{\text{prop}} = 0.5$ and $M_{\text{gap}} = 15$. We set each weight γ_q to be the average quantization error for the q -th codebook computed over 150 random training samples, normalized to sum to 1.0 over all codebooks. This gave a small improvement on validation data compared to using uniform γ_q values. For feature extraction, we found that the last layer of the encoder gave the best results on validation data. Our largest model was trained in 5 days using 2 RTX 3090 GPUs.

3.2. Downstream evaluation

We evaluated our models following the HEAREval procedure [16] and a subset of its instance-level tasks. The embedding for each instance is obtained by averaging the frame-level activations from the last encoder layer. The resulting embedding is fed to a multilayer perceptron and a grid search over hyperparameters is performed as described in [16]. The upstream model is frozen, so only the downstream model parameters are trained for each evaluation task. For music, we evaluate in the 50-hour subset of the NSynth Pitch [26] dataset (NS), which consists of instrument notes and where the goal is to classify into 88 possible pitch values. We also evaluate on music genre classification (GC), using the GTZAN Genre dataset [27], which consists of 1000 30-second audio tracks categorized into 10 genres. For speech, we evaluate on the Google Speech Commands dataset v2 (SC) [28], where each utterance has to be classified into 10 possible commands or noise or unknown, and on the CREMA-D dataset [29] for 6-class emotion recognition (ER). For environmental sounds, we evaluate on the Freesound Dataset 50K (FSD) [30], which contains 100 hours of sound events labelled with one or more of 200 possible classes from the Audioset ontology. We also evaluate in the ESC-50 dataset (ESC) [31], a collection of 2000 audio samples each with a single acoustic event belonging to one of 50 possible classes.

We report accuracy for all tasks except FSD, where mean average precision (mAP) is used, and we conduct bootstrapping with 100 iterations obtaining confidence intervals as the 2.5 and 97.5 percentiles [32]. We also report a global metric as in [16] by normalizing the performance on each task, clipping the resulting value to $[-1, 1]$ and then taking the average of these values across tasks. The normalization is done by subtracting the mean and dividing by the standard deviation of the performance of each task on the HEAREval 2021 benchmark results. A value of +1 means that the performance of the system is at least one standard deviation better than the average performance on every evaluated task.

We also evaluated our models in ASR following the SUPERB protocol [33], which consists of a vanilla 2-layer 1024-unit BiLSTM optimized by CTC loss on characters. This downstream model receives sequences of embeddings from each EnCodecMAE layer and learns a weighted average to combine them into a unique input sequence. We halved the frame-rate of our embeddings by averaging every 2 contiguous embeddings to speedup the training. The model is trained in Librispeech 100-hours of clean speech and decoding is done using the Librispeech official 4-gram language model.

4. Results

Table 1 shows the results for various configurations of our proposed model and state-of-the-art models from recent literature, for the six tasks described in Section 3.2. Comparing different feature extractors we can see that they have a large effect in downstream performance and that the impact is different across tasks. Globally, melspectrograms yielded better results than EnCodec representations (lines 2 and 3 in Table 1) with 95.9 vs 83.4 global performance. However, for the task of pitch prediction (NS), our models using EnCodec outperformed melspectrograms and an ensemble system (line 18) which concatenates embeddings from lines 16 and 17 and CREPE [35], a model trained for pitch prediction using annotated datasets. This result highlights the importance of the input representation. It is also worth noting that the input representations alone do not yield good results with a global metric of -76.6 for EnCodec and -66.1 for melspectrograms.

We also investigated the influence of the datasets used for pretraining on task performance (lines 3 through 6). Predictably, pretraining with speech-only data (LL6K Only) led to decreased performance in music and environmental tasks, while pretraining with music-only data (FMA Only) resulted in lower performance in speech tasks. Models pretrained on Audioset, or a mixture of Audioset, FMA, and Libri-Light, exhibited more balanced performance across music, speech, and environmental tasks. Pretraining with Audioset alone gave a better performance in environmental tasks but performed slightly worse than the mixture in speech tasks.

Scaling up the model size gave consistent improvements in global performance (lines 1, 3 and 9). Further, self-training (lines 3 vs 7, 4 vs 8, and 9 vs 10) enhanced our model’s performance across most tasks. This finding aligns with prior research [10, 7], reinforcing the effectiveness of self-training techniques in improving SSL representations.

BEATs [7], BYOL-A [15], AudioMAE [4] and MSM-MAE-512 [21] are models pretrained in Audioset. Our base Audioset-trained model with self-training (line 8) outperforms these models on music and speech tasks and performs similarly on environmental task. Our main differences with BEATs are the initial target definition (EnCodec in our model, random quantizers in BEATs), and the input signal format (frames in our model, rectangular patches in BEATs). Previous works [21, 5, 1] found that using frames instead of rectangular patches improved results in speech-related tasks while degrading it in event detection tasks. The improvement in speech tasks is likely because rectangular patches span multiple frames and a frequency band, losing temporal resolution which might be useful for speech and tasks that require analyzing a signal over time. Avoiding rectangular patching, which is borrowed from computer vision research [36], and using frames instead, aligns better with traditional audio representations like spectrograms. In general, Table 1 shows that our model reaches significantly

¹https://huggingface.co/facebook/encodec_24khz

	Music		Speech		Env		Global	Params(M)	Pretrain Data
	NS	GC	SC	ER	FSD	ESC			
(1) Mel→EC (Small)	85.9±1.2	87.5±1.9	95.2±0.7	74.8±0.9	48.5±1.2	80.1±1.7	94.8	43.6	AS+FMA+LL6K
(2) EC→EC (Base)	91.7±1.0	85.6±1.9	92.2±1.0	72.0±1.0	44.1±1.3	74.6±2.1	83.4	94.0	AS+FMA+LL6K
(3) Mel→EC (Base)	84.6±1.0	87.6±1.9	96.3±0.5	75.5±1.0	49.5±1.1	79.8±1.8	95.9	86.6	AS+FMA+LL6K
(4) AS only	86.1±1.1	87.1±1.9	93.6±0.7	73.8±1.1	51.7±1.3	81.5±2.1	97.0	86.6	AS
(5) FMA only	87.7±1.1	89.2±2.0	87.4±0.9	68.7±1.1	48.5±1.3	80.4±2.2	89.1	86.6	FMA
(6) LL6K only	83.4±1.3	83.4±2.0	95.4±0.6	75.6±1.1	41.3±1.2	68.8±1.6	75.3	86.6	LL6K
(7) Mel→EC (Base + ST)	81.7±1.4	87.8±2.1	96.9±0.5	76.3±1.0	50.7±1.2	81.6±2.1	97.6	86.6	AS+FMA+LL6K
(8) AS only	83.5±1.3	87.7±2.3	94.6±0.6	75.1±1.1	52.9±1.1	84.6±1.6	99.2	86.6	AS
(9) Mel→EC (Large)	85.3±1.4	85.8±2.2	96.4±0.5	77.1±0.9	50.3±1.1	80.2±1.9	97.1	261.6	AS+FMA+LL6K
(10) Mel→EC (Large + ST)	83.4±1.0	87.2±2.1	97.0±0.4	77.2±0.9	51.0±1.0	84.1±1.8	100.0	261.6	AS+FMA+LL6K
(11) AS only	86.4±1.3	86.4±1.7	94.4±0.7	75.7±0.9	53.4±1.0	86.0±1.8	99.1	261.6	AS
(12) BEATS (Iter 1) [7]	82.4±1.3	85.4±2.4	91.0±0.8	68.7±1.1	50.6±0.9	82.3±1.7	94.4	90	AS
(13) BEATS (Iter 3) [7]	82.9±1.0	86.2±2.9	90.4±0.8	69.0±1.1	53.5±1.1	85.1±1.5	96.0	90	AS
(14) BYOL-A [15]	68.3±1.4	86.0±2.0	93.1±0.7	65.9±1.1	51.3±0.9	83.6±2.0	82.6	5.3	AS
(15) AudioMAE (PT) [4]	67.2±1.5	72.0±3.1	45.2±1.6	39.3±1.0	37.6±1.2	58.5±2.1	-32.1	86	AS
(16) MSM-MAE-512 [21] †	81.2	86.1	86.4	73.4	52.2	85.6	91.7	86	AS
(17) Fuse HuBERT [34] †	68.8	79.6	95.7	75.2	41.3	74.3	62.9	1000	LL60K
(18) Fuse Wav2Vec2 [34] †	60.6	79.3	96.9	69.2	40.3	69.5	51.4	317	LV60K

Table 1: Performance in a subset of tasks from HEAREval benchmark. The first block corresponds to our models, the second to models from recent literature pretrained in an unsupervised way. We name our models with the syntax Input→Target, e.g., Mel→EC takes melspectrograms as input and predicts EnCodec tokens. ST stands for self-training. For lines with a †, we show the HEAREval leaderboard results or the ones reported in the cited work. For BEATS, BYOL-A and AudioMAE we used the official weights and adapted the implementation to be used with HEAREval. Reported deviations are the difference between the metric and the furthest interval bound calculated with bootstrapping.

Model	w/o	w/LM	Params(M)
Wav2Vec [37]	15.86	11.00	32.5
VQ-Wav2Vec [38]	17.71	12.80	34.1
DeCOAR 2.0 [39]	13.02	9.07	89.8
Wav2Vec 2.0 Base [12]	6.43	4.79	95.0
Wav2Vec 2.0 Large [12]	3.75	3.10	317.4
HuBERT Base [10]	6.42	4.79	94.7
HuBERT Large [10]	3.62	2.94	316.6
Mel→EC (Base)	15.53±1.29	10.42±1.04	86.6
LL6K only	14.91±1.21	10.12±0.96	86.6
Mel→EC (Base+ST)	14.41±1.28	9.90±0.99	86.6
LL6K only	13.72±1.28	9.29±0.96	86.6
Mel→EC (Large+ST)	12.44±0.99	8.59±0.79	261.6

Table 2: Word Error Rates (WER) obtained in the SUPERB ASR downstream task. We report metrics with and without a language model and their deviations using bootstrap conditioned by speaker and video segment.

better performance on speech tasks compared to patch-based models (compare line 4 with 12,15 and 16 and line 8 with 13) while performing on par with patch-based models in environmental tasks. This might be partially explained by our use of EnCodec instead of random quantizers or melspectrograms as targets.

Fuse HuBERT and Fuse Wav2Vec 2.0 [34] generate embeddings by averaging the activations from all the transformer layers of HuBERT XL [10] and Wav2Vec 2.0 Large [12] models, respectively. These models are pretrained solely on speech data (60K hours of Libri-Light) with a focus in learning good representations for ASR. Our base model trained with Libri-Light (line 6) performs better than these models across all non-speech tasks except for ESC, despite having less parameters and being trained in a dataset an order of magnitude smaller. Moreover, our large model trained with the mixture of datasets and self-training (line 10) gives better performance across all tasks,

including the speech tasks, while still being a smaller model trained on less data.

Given the strong performance of our models in speech-related tasks, we further evaluated them in the more challenging task of ASR. The results in Table 2 indicate that our models still fall short of the performance achieved by the most recent speech SSL models such as HuBERT and Wav2Vec 2.0. Nevertheless, the performance is comparable to older SSL models like DeCOAR 2.0 and Wav2Vec. While pretraining only with speech data provides some improvement, the key to achieving HuBERT-level performance in ASR may not only be the dataset but also the target definition, as clusters of MFCCs might be highly correlated with phonemes, and the input representation, which HuBERT learns from the raw waveform using a CNN.

5. Conclusions

We introduce EnCodecMAE, a novel universal audio representation model trained on masked inputs to predict targets from EnCodec, a neural audio codec. Our models, pretrained with Audioset or a mixture of datasets using self-training, achieve the best global performance across speech, music and environmental tasks. While pretraining only on Audioset results in a strong model, adding more speech leads to better results for speech-related tasks. Overall, our results show a clear trade-off between tasks depending on the input representation and the pretraining dataset. We also achieve a decent performance in ASR, comparable to DeCOAR 2.0. In future works we will focus in the challenging task of improving the performance in ASR without sacrificing performance in music and environmental tasks by exploring different training targets. Finally, given that our models are not fine-tuned and use light-weight downstream models, they allow for efficient implementation of multiple audio tasks in parallel with a single backbone.

6. Acknowledgements

This project has received funding from the European Union's Horizon 2020 research and innovation programme under the Marie Skłodowska-Curie grant agreement No 101007666. This work was also supported by a Google Faculty Research Award, 2019, and an Amazon Research Award, 2019.

7. References

- [1] Y. Gong, C.-I. Lai, Y.-A. Chung, and J. Glass, "Ssast: Self-supervised audio spectrogram transformer," in *AAAI Conference on Artificial Intelligence*, 2022.
- [2] L. Wang *et al.*, "Towards learning universal audio representations," in *ICASSP 2022*.
- [3] A. Seth, S. Ghosh, S. Umesh, and D. Manocha, "Slicer: Learning universal audio representations using low-resource self-supervised pre-training," in *ICASSP 2023*.
- [4] P.-Y. Huang, H. Xu, J. Li, A. Baevski, M. Auli, W. Galuba, F. Metze, and C. Feichtenhofer, "Masked autoencoders that listen," in *NeurIPS 2022*.
- [5] A. Baade, P. Peng, and D. Harwath, "Mae-ast: Masked autoencoding audio spectrogram transformer," *Interspeech 2022*.
- [6] D. Chong, H. Wang, P. Zhou, and Q. Zeng, "Masked spectrogram prediction for self-supervised audio pre-training," in *ICASSP 2023*.
- [7] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, and F. Wei, "Beats: Audio pre-training with acoustic tokenizers," in *ICML 2023*, June 2023.
- [8] K. Koutini, J. Schlüter, H. Eghbal-zadeh, and G. Widmer, "Efficient training of audio transformers with patchout," in *Interspeech 2022*.
- [9] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio Spectrogram Transformer," in *Interspeech 2021*.
- [10] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhota, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM TASLP*, 2021.
- [11] A. Baevski and A. Mohamed, "Effectiveness of self-supervised pre-training for asr," in *ICASSP 2020*.
- [12] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *NeurIPS 2020*.
- [13] A. Jansen, M. Plakal, R. Pandya, D. P. Ellis, S. Hershey, J. Liu, R. C. Moore, and R. A. Saurous, "Unsupervised learning of semantic audio representations," in *ICASSP 2018*.
- [14] J. Shor, A. Jansen, R. Maor, O. Lang, O. Tuval, F. d. C. Quitry, M. Tagliasacchi, I. Shavitt, D. Emanuel, and Y. Haviv, "Towards learning a universal non-semantic representation of speech," *Interspeech 2020*.
- [15] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, "Byol for audio: Self-supervised learning for general-purpose audio representation," in *IJCNN 2021*.
- [16] J. Turian *et al.*, "Hear: Holistic evaluation of audio representations," in *NeurIPS 2021*.
- [17] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*, 2019.
- [18] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.
- [19] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *CVPR 2022*.
- [20] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM TASLP*, 2021.
- [21] D. Niizumi, D. Takeuchi, Y. Ohishi, N. Harada, and K. Kashino, "Masked spectrogram modeling using masked autoencoders for learning general-purpose audio representation," in *HEAR: Holistic Evaluation of Audio Representations*. PMLR, 2022, pp. 1–24.
- [22] S. Xie, H. Zhang, J. Guo, X. Tan, J. Bian, H. H. Awadalla, A. Menezes, T. Qin, and R. Yan, "Residual: Transformer with dual residual connections," *arXiv preprint arXiv:2304.14802*, 2023.
- [23] J. F. Gemmeke, D. P. W. Ellis, D. Freedman, A. Jansen, W. Lawrence *et al.*, "Audio set: An ontology and human-labeled dataset for audio events," in *ICASSP 2017*.
- [24] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, "FMA: A dataset for music analysis," in *ISMIR 2017*.
- [25] J. Kahn, M. Rivière, W. Zheng, E. Kharitonov, Q. Xu, P. E. Mazaré, J. Karadayi, V. Liptchinsky, R. Collobert, C. Fuegen, T. Likhomanenko, G. Synnaeve, A. Joulin, A. Mohamed, and E. Dupoux, "Libri-light: A benchmark for asr with limited or no supervision," in *ICASSP 2020*.
- [26] J. Engel, C. Resnick, A. Roberts, S. Dieleman, M. Norouzi, D. Eck, and K. Simonyan, "Neural audio synthesis of musical notes with wavenet autoencoders," in *ICML 2017*.
- [27] G. Tzanetakis and P. Cook, "Musical genre classification of audio signals," *IEEE Transactions on speech and audio processing*, 2002.
- [28] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," *arXiv preprint arXiv:1804.03209*, 2018.
- [29] H. Cao, D. G. Cooper, M. K. Keutmann, R. C. Gur, A. Nenkova, and R. Verma, "Crema-d: Crowd-sourced emotional multimodal actors dataset," *IEEE transactions on affective computing*, 2014.
- [30] E. Fonseca, X. Favory, J. Pons, F. Font, and X. Serra, "Fsd50k: an open dataset of human-labeled sound events," *IEEE/ACM TASLP*, 2021.
- [31] K. J. Piczak, "ESC: Dataset for Environmental Sound Classification," in *Proceedings of the 23rd Annual ACM Conference on Multimedia*, 2015.
- [32] L. Ferrer and P. Riera, "Confidence Intervals for evaluation in machine learning." [Online]. Available: <https://github.com/luferrer/ConfidenceIntervals>
- [33] S.-w. Yang, P.-H. Chi, Y.-S. Chuang, C.-I. J. Lai, K. Lakhota, Y. Y. Lin, A. T. Liu, J. Shi, X. Chang, G.-T. Lin *et al.*, "Superb: Speech processing universal performance benchmark," *Interspeech 2021*.
- [34] T.-Y. Wu, T.-Y. Hsu, C.-A. Li, T.-H. Lin, and H.-y. Lee, "The efficacy of self-supervised speech models for audio representations," in *HEAR: Holistic Evaluation of Audio Representations*, 2022.
- [35] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, "Crepe: A convolutional representation for pitch estimation," in *ICASSP 2018*.
- [36] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.
- [37] S. Schneider, A. Baevski, R. Collobert, and M. Auli, "wav2vec: Unsupervised pre-training for speech recognition," *Interspeech 2019*.
- [38] A. Baevski, S. Schneider, and M. Auli, "vq-wav2vec: Self-supervised learning of discrete speech representations," *ICLR 2020*.
- [39] S. Ling and Y. Liu, "Decoar 2.0: Deep contextualized acoustic representations with vector quantization," 2020.